ML4SCI GSoC 2026 — CMS E2E Evaluation Tasks

Task 2b (Super-Resolution) and Task 2c (Agentic HEPTAPOD)

Documented in repository `CMS_E2E`

March 21, 2026

Contents

1 Task 2b — Super-resolution at the CMS detector **1**
- 1.1 Objective . . . 1
- 1.2 Dataset and labels . . . 1
- 1.3 Model . . . 1
- 1.4 Training and protocols . . . 2
- 1.5 Evaluation . . . 2
- 1.6 Reference results (checked in) . . . 2
- 1.7 Code layout . . . 2

2 Task 2c — Agentic AI for HEP analyses (HEPTAPOD) **2**
- 2.1 Objective . . . 2
- 2.2 Fork and branch . . . 2
- 2.3 Proposed layout (`cms_agent/`) . . . 3
- 2.4 Tool inventory . . . 3
- 2.5 Minimal working adapter . . . 3
- 2.6 Compliance . . . 3

Scope

This report summarizes work for two **independent** ML4SCI umbrella evaluation tracks: **Task 2b** (generative super-resolution on CMS-style jet images) and **Task 2c** (proposed HEPTAPOD layout and tooling for agentic CMS analyses). Each section is self-contained.

1 Task 2b — Super-resolution at the CMS detector

1.1 Objective

Train a **Generative Adversarial Network** to map low-resolution calorimeter jet images to high-resolution targets, using the official ML4SCI parquet release (CERNBox link in the GSoC PDF). The submission also documents modeling and optimization choices as required by the task statement.

1.2 Dataset and labels

- **Low resolution:** `X_jets_LR`, shape $3 \times 64 \times 64$.
- **High resolution:** `X_jets`, shape $3 \times 125 \times 125$.

- **Class label:** `y` $\in \{0, 1\}$ (quark vs. gluon), stratified in train/validation/test splits (default 80%/10%/10%).

The public shards use a **single Parquet row group per file**; the loader therefore **streams record batches** and materializes only the row indices needed for training, avoiding full-file loads per sample.

1.3 Model

- **Generator (SRGAN-style):** bicubic upsampling of LR to 125×125 as an explicit baseline, plus a residual CNN (residual blocks, InstanceNorm) predicting a correction added to that baseline.
- **Discriminator (PatchGAN):** convolutional stack with optional **class conditioning** via embedding-modulated feature maps (quark/gluon).
- **Losses:** least-squares GAN objectives; **L1** reconstruction on normalized HR; small weight on **adversarial** loss; optional **energy-matching** term on denormalized total deposited energy ($\sum \max(0, x)$ over channels and pixels).

1.4 Training and protocols

Two tiers are supported in code and documentation:

1. **Host-constrained (reference):** materialized subsets via `-max-train-samples` / `-max-val-samples`, fewer epochs, optional `-memmap-train` when RAM is insufficient for full tensors.
2. **Full-scale:** no subset caps, `-epochs 20`, `-memmap-train` as needed for ∼full training index sets.

Validation logs `val_l1`, bicubic baseline `val_bicubic_l1`, and `val_energy_mae` to `history.csv`. Checkpoints: `checkpoint_best.pt` (best val L1), `checkpoint_last.pt`.

1.5 Evaluation

Script `super_resolution/evaluate.py` reports JSON metrics on the test split (full test by default), including SR vs. HR and a **bicubic** baseline: PSNR/SSIM, mean total-energy ratio, energy MAE, mean radial-profile L1, and a toy linear probe on per-image channel means. Results are written as structured JSON (e.g. `metrics_report.json`).

1.6 Reference results (checked in)

Using the **full_run** configuration (subset caps, 8 epochs, $\lambda_{\text{energy}} = 0.05$) on the full test split ($n = 13\,932$), illustrative numbers from `super_resolution/runs/full_run/metrics_report.json`:

Metric	SR	Bicubic
PSNR mean (dB)	44.28	44.28
SSIM mean	0.9935	0.9934
Energy MAE vs. HR	1.20	21.29
Mean radial profile L1 vs. HR	0.00040	0.00132

The energy penalty materially improves **energy fidelity** vs. bicubic; image PSNR/SSIM remain close to the trivial resize because the task geometry is dominated by a smooth upscale.

An **ablation** with `-lambda-energy 0` (`ablation_no_energy/`) shows degraded energy scale (e.g. energy MAE ≈ 153 vs. HR) despite similar PSNR, illustrating the need for physics-aware terms beyond pixel loss.

1.7 Code layout

`super_resolution/data.py`, `models.py`, `train.py`, `evaluate.py`; narrative in `MODEL_DISCUSSION.md` and `README.md`.

2 Task 2c — Agentic AI for HEP analyses (HEPTAPOD)

2.1 Objective

Per the evaluation brief: clone HEPTAPOD, work on a **separate branch** in your fork, propose an **alternative folder structure** and **tool set** informed by repositories linked on the project page, and **do not open a pull request** to the upstream HEPTAPOD repository unless the program explicitly requests it.

2.2 Fork and branch

Development branch: `gsoc26-cms-restructure`. Example clone:

```
git clone -b gsoc26-cms-restructure \
  https://github.com/KenWuqianghao/heptapod.git heptapod
```

Upstream reference: https://github.com/tonymenzo/heptapod.

2.3 Proposed layout (`cms_agent/`)

A parallel tree separates CMS-oriented orchestration from the existing generic `tools/`, `llm/`, and `examples/` directories:

- `cms_agent/agents/` — planner contract and role boundaries.
- `cms_agent/workflows/` — example run cards and (where applicable) JSON schema for validated planner output.
- `cms_agent/tools/adapters/` — thin adapters toward CMSSW, `coffea`, `awkward`, etc., preserving HEPTAPOD-friendly JSON outputs.
- `cms_agent/configs/` — model routing and tool allowlists (e.g. forbid arbitrary shell).
- `cms_agent/knowledge/` — curated snippets / RAG manifest placeholders.
- `cms_agent/eval/` — golden prompts and small fixtures (e.g. histogram contract checks).

Architecture narrative: `heptapod/ARCHITECTURE_CMS_AGENT.md`.

2.4 Tool inventory

`cms_agent/TOOL_INVENTORY.md` maps each conceptual tool to dependencies cited in HEPTAPOD documentation (Orchestral AI runtime, MCP bridges, PDG/INSPIRE tools in-repo, CMSSW/coffea-class stacks for CMS-specific steps) and records failure modes.

2.5 Minimal working adapter

`cms_agent/tools/adapters/cms_histograms.py` implements a **NumPy-only** leading-p_T histogram JSON contract so the scaffold can be exercised without CMSSW or `coffea` installed.

2.6 Compliance

Work remains on the contributor fork branch; no merge to upstream was required for the evaluation checklist.

Repository map

Task 2b: `super_resolution/`, run artifacts under `super_resolution/runs/`.
Task 2c: HEPTAPOD fork (clone separately; see `README.md` in this repo).
This PDF: `docs/GSoC26_CMS_E2E_Report.pdf` (build from `docs/GSoC26_CMS_E2E_Report.tex`).